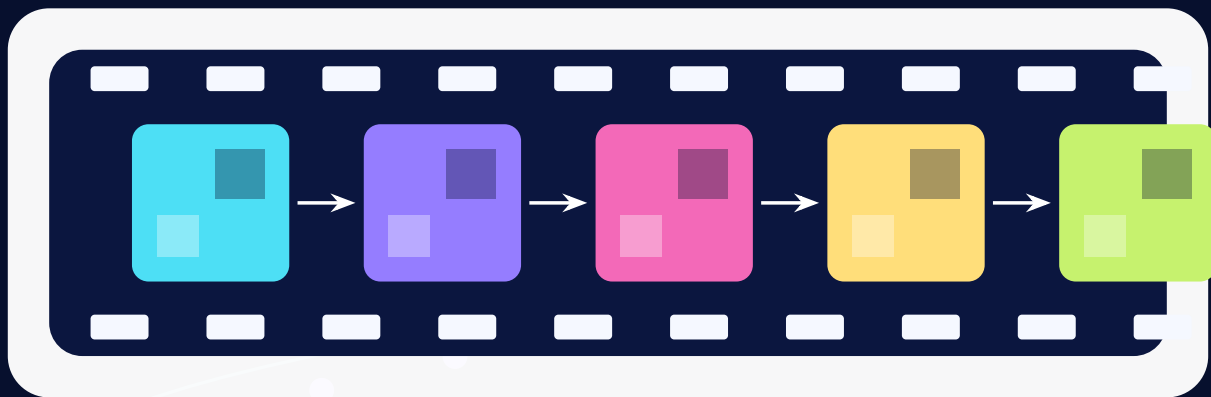


映像圧縮のひみつを、図と数式で冒険しよう

小学生でもわかる VCCコーデック

名前だけ VCC ・ 中身は VVC / H.266



予測する・差をとる・変身させる・丸める・短い記号にする

ILLUSTRATED EDITION

小学生でもわかる VCC コーデック

名前だけ VCC。中身は正しい VVC / H.266

VCC と VVC、 どっちなの？

この本では、表紙の名前だけ **VCC**。
正式な技術名は **VVC** です。

VVC

H.266

最初に、名前の整理

国際標準の正式な略し方は、**VVC = Versatile Video Coding**（バーサタイル・ビデオ・コーディング）です。規格番号は**H.266**。この本は依頼された題名を大切に
して、表紙では「VCC」と書きますが、本文では正しい名前「VVC」を使います。

ここだけ覚えよう

コーデックとは、動画を小さくたたむ**エンコーダ**と、もとに近い動画へ広げる**デコーダ**の組み合わせです。VVC は、その新しい世代のルールです。

この本の約束

むずかしい技術を、絵 → ことば → 数式の順にほどきます。

1

まず絵から

むずかしい言葉は、図と小さな例で先にイメージする。

2

式を日本語で

数式は、記号の意味と「何をする式か」まで読む。

3

単純化は正直に

本物はもっと複雑。大切な中心を、うそにならない形で学ぶ。

4

図はオリジナル

規格図の転載ではなく、この本のための学習用図解を使う。

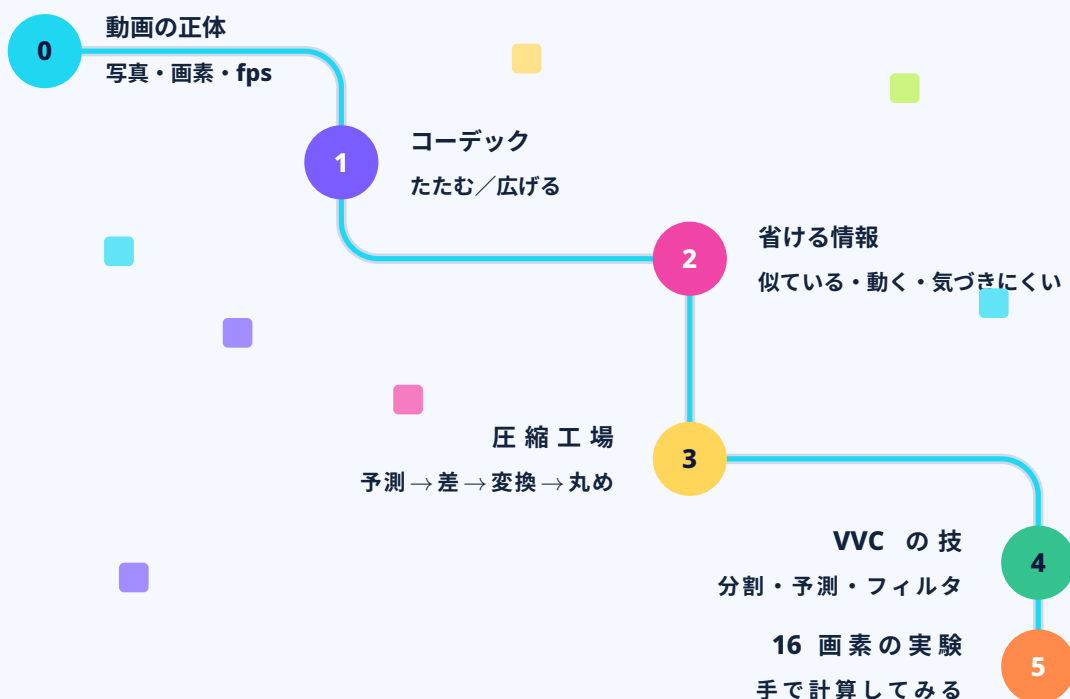
わからない式は、いったん飛ばして大丈夫！

図と「ここだけ覚えよう」だけでも、VVC の圧縮ストーリーはつかめます。あとで戻ると、数式が絵の短い言い方に見えてきます。

対象：小学4年生くらいから大人まで。かけ算・引き算がわかれば読み始められます。

この本の冒険マップ

左から右へ進むと、動画が小さな「ビット列」へ変身します。



読み方は自由！

まず絵だけ

図と「ここだけ覚えよう」を追う。

しくみまで

数式の箱にも寄り道する。

技術者モード

「もっと知りたい人へ」も読む。

最初のなぞ：動画は、ファイルをただ押しつぶすものではありません。**予想できるところを見つけ、必要な差だけを書くのが圧縮の中心です。**

目次

第 1 章	動画は「写真の高速パラパラまんが」	1
1.1	1 秒の中に、写真が何枚もある	1
1.2	写真は、小さな色のタイルでできている	2
1.3	そのまま保存すると、どれくらい大きい？	3
第 2 章	コーデックは「たたむ人」と「広げる人」	5
2.1	動画を小さくして、また見られる形にする	5
2.2	「箱」と「中身」は別もの	6
2.3	完全にもどす圧縮、少しだけ変えて小さくする圧縮	7
第 3 章	動画には「書かなくても予想できるもの」が多い	8
3.1	近くの画素は、たいてい似ている	8
3.2	前のフレームと、たいていよく似ている	9
3.3	人の目は、明るさにとても敏感	9
3.4	4:2:0 は、色のメモを間引く方法	10
第 4 章	コーデック工場の 6 つの工程	12
4.1	エンコーダの中を、上から見てみよう	12

4.2	工程 1：予測する	13
4.3	工程 2：予測との差（残差）を取る	13
4.4	工程 3・4：変換して、丸める	13
4.5	工程 5：よく出る記号を短く書く	14
4.6	工程 6：自分でも再生して、次の予測に使う	14
第 5 章	引き算から始める、圧縮の数学	15
5.1	まずは 1 画素：本物－予測＝残差	15
5.2	4×4 画素でも、やることは同じ	16
5.3	変換は「並び方」を変えて、エネルギーを集める	17
5.4	量子化は「ものさしの目を粗くする」	18
5.5	デコーダは逆向きに進む	18
第 6 章	VVC のすごさ 1：形に合わせて細かく分ける	20
6.1	最初は大きな CTU から	20
6.2	QT・BT・TT という切り方	21
6.3	エンコーダは「候補」をくらべる	22
第 7 章	VVC のすごさ 2：予測の引き出しが多い	24
7.1	イントラ予測：となりの画素から中を想像する	24
7.2	MIP：少ない参照画素から、行列で面を作る	25
7.3	インター予測：前後のフレームから動きを探す	26
7.4	平行移動だけでなく、回転や拡大も近似する	27
7.5	予測候補を、賢く使い回す	27

第 8 章	VVC のすごさ 3：残差に合う変換と、短い符号	29
8.1	MTS：変換にも選択肢がある	29
8.2	LFNST：低い周波数どうしを、もう一度まとめる	30
8.3	SBT：残差がある部分だけを変換する	31
8.4	量子化：0 を増やすが、誤差も管理する	31
8.5	CABAC：出やすさを予想して、短く書く	32
第 9 章	VVC のすごさ 4：再生画像をみがき、いろいろな映像に対応する	34
9.1	なぜフィルタが必要なの？	34
9.2	3 段階の代表的な仕上げ	35
9.3	CC-ALF：明るさを手がかりに、色を直す	36
9.4	LMCS：明るさの使い方を映像に合わせる	36
9.5	「Versatile」は、いろいろな映像に使えるという意味	37
9.6	サブピクチャ：画面の一部をひとまとまりとして扱う	37
第 10 章	VVC はどれくらい小さくできる？	39
10.1	代表的な目標：HEVC の約半分のビットレート	39
10.2	ビットレートから、だいたいのファイルサイズを計算する	40
10.3	「同じ画質」はどうやって比べる？	40
10.4	軽さの代わりに、計算は重くなりやすい	42
10.5	3 世代を並べると、進化が見える	43
第 11 章	16 画素で、圧縮を手計算してみよう	44
11.1	前のフレーム：明るい四角が中央にある	44

11.2	次のフレーム：四角が右へ1画素動いた	45
11.3	予測ブロックを作る	46
11.4	残差を引き算する	46
11.5	完全一致しないとき：1画素だけ92だったら	48
11.6	量子化の細かさで、結果が変わる	48
11.7	この小さな実験に、VVCの全体が入っている	49
第12章	まとめ：VVCを30秒で説明しよう	50
12.1	動画がビット列になるまで	50
12.2	VVCの「すごい」を4枚のカードで	50
12.3	3つの長さで説明してみよう	51
12.4	よくある質問	53
12.5	卒業クイズ：全10問	54
ことば図鑑		56
参考資料と、正確さについて		58
おわりに		60

MISSION 1

動画は「写真の高速パラパラまんが」

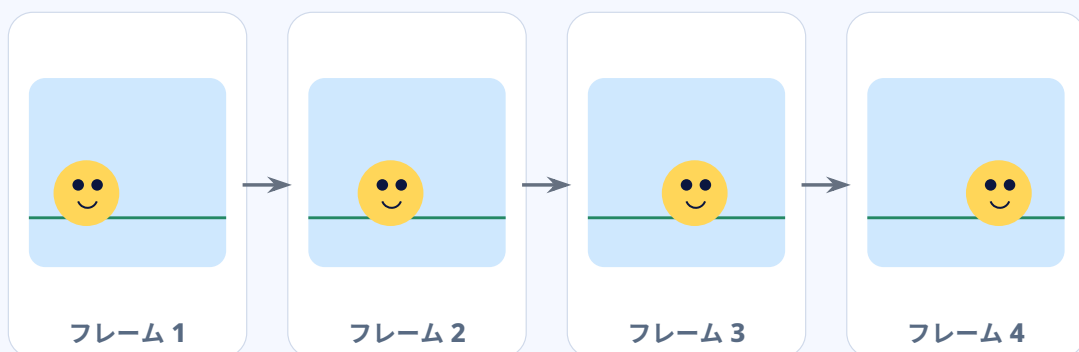
最初のミッション

動画の正体を、**フレーム**・**画素**・**fps**の3語で説明できるようになろう。

1.1 1秒の中に、写真が何枚もある

動画は、ほんとうに少しずつ違う写真を高速で見せています。1枚1枚の写真を**フレーム**と呼び、1秒間に何枚見せるかを**fps** (frames per second) で表します。

動画は「写真の高速パラパラまんが」



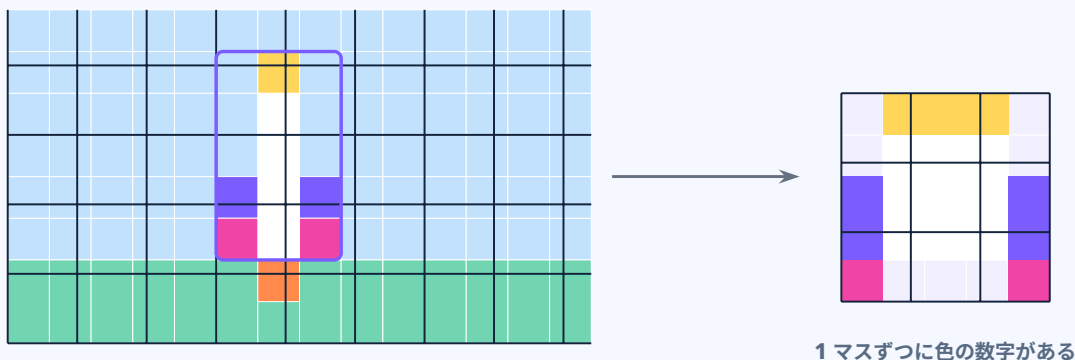
30 fps なら、1 秒に 30 枚

ここだけ覚えよう

フレームは動画の 1 枚の写真。30 fps なら、1 秒間に 30 枚の写真が流れます。

1.2 写真は、小さな色のタイルでできている

画面をものすごく拡大すると、色のついた小さな点が並んでいます。これが画素（ピクセル）です。たとえば 4K 動画は、横 3840 個、縦 2160 個ほどの画素を 1 フレームに持ちます。



ちょっと数学

横の画素数を W 、縦を H とすると、1 フレームの画素数は

$$\text{画素数} = W \times H$$

4K なら $3840 \times 2160 = 8,294,400$ 。たった 1 枚で約 829 万画素です。

1.3 そのまま保存すると、どれくらい大きい？

まずは単純に、1 画素を赤・緑・青の 3 色で表し、各色に 1 バイトずつ使うと考えます。すると 1 画素は 3 バイトです。

横の画素数
 W

×

縦の画素数
 H

×

1 画素
3 バイト

×

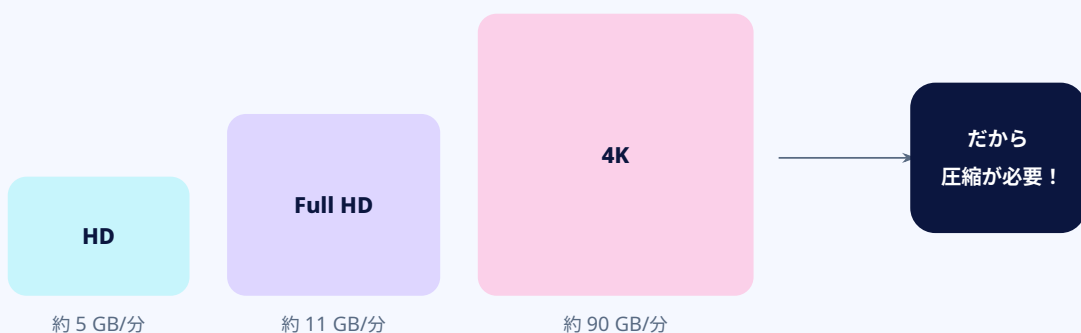
1 秒の枚数
fps

$$1 \text{ 秒のデータ量} = W \times H \times 3 \times \text{fps}$$

表1.1: 圧縮しない RGB 動画の大きさ（おおよその値）

画面	フレームレートの設定	1 秒のデータ量	1 分間のデータ量
HD (1280×720)	30	約 83 MB	約 5.0 GB
Full HD (1920×1080)	30	約 187 MB	約 11.2 GB
4K (3840×2160)	60	約 1.49 GB	約 89.6 GB

動画は「写真の高速パラパラまんが」



ほんとうの話

実際のカメラや動画サービスは、色の表し方や画素あたりの細かさがさまざまです。この表は「なぜ圧縮が必要か」をつかむための、わかりやすい計算です。

ミニクイズ

4K・60 fps の動画を、そのまま 10 分保存すると約何 GB でしょう？

答え：約 $89.6 \times 10 = 896$ GB。ほぼ 1 TB です。

MISSION 2

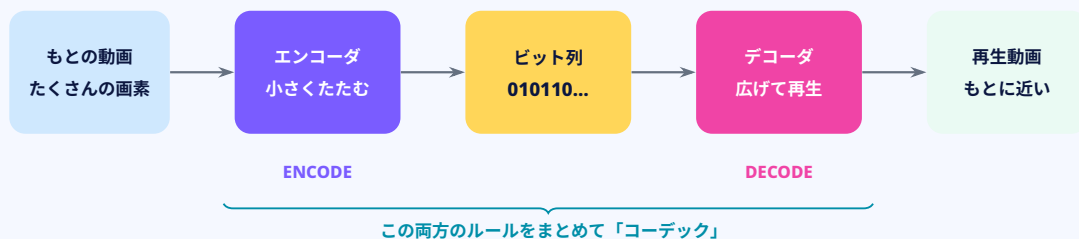
コーデックは「たたむ人」と「広げる人」

この章のゴール

エンコード、ビットストリーム、デコードの流れを、人に説明できるようになる。

2.1 動画を小さくして、また見られる形にする

圧縮前の動画は、とても大きいデータです。そこで送る前や保存する前に、小さな記号の列へ変えます。この作業を**エンコード**と呼びます。見るときには、その記号から映像を組み立て直します。これが**デコード**です。



COrder + DECoder = CODEC

ここだけ覚えよう

コーデックは、動画を小さなデータにする方法と、そのデータから動画を再生する方法のセットです。

2.2 「箱」と「中身」は別もの

動画ファイルの名前には、.mp4 や .mkv がつくことがあります。これは主に**コンテナ**、つまり映像・音声・字幕などを入れる「箱」の種類です。箱の中の映像が、どのコーデックで圧縮されているかは別の話です。

MP4 という箱

映像 : VVC / H.266

音声

字幕

≠

VVC という中身

画素の予測や変換のルール

010011101...

まちがえやすい

「MP4 だから VVC」「VVC だから拡張子は必ず決まったもの」とは限りません。**箱（コンテナ）**と**中身（コーデック）**を分けて考えます。

2.3 完全にもどす圧縮、少しだけ変えて小さくする圧縮

種類	どうなる?	例
可逆圧縮	1 ビットも変えず、完全にもとへ戻せる。一般には動画をあまり小さくできない。	文書の ZIP、画像の PNG など
非可逆圧縮	人が気づきにくい細部を少し丸めて、大きく小さくする。	配信動画の多く、VVC など

可逆

100 個 → 100 個を完全再現

✓ 完全一致

非可逆

100 個 → 大切な特徴を再現

✗ 画素は完全一致しない

ほんとうの話

VVC は、主に**非可逆圧縮**として使われます。ただし「ただ雑に消す」のではありません。人の目、画像の形、前後の動きを利用して、目立ちにくいところへ誤差を集めます。

ミニクイズ

movie.mp4 という名前だけを見て、中の映像コーデックを必ず言い当てられる？

答え：言い当てられません。MP4 は箱の種類だからです。

MISSION 3

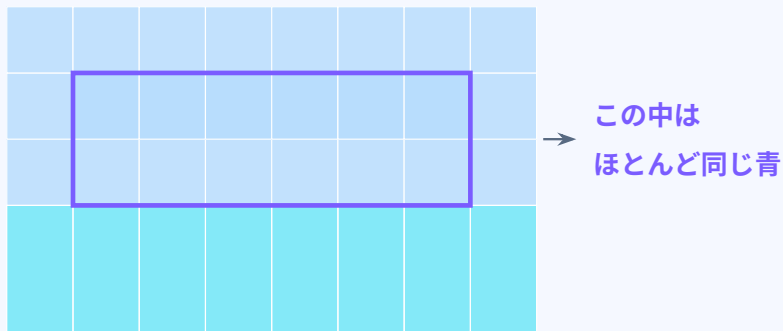
動画には「書かなくても予想できるもの」が多い

圧縮のひみつ

動画を小さくできる理由は、似ている・動いている・気づきにくいの3つです。

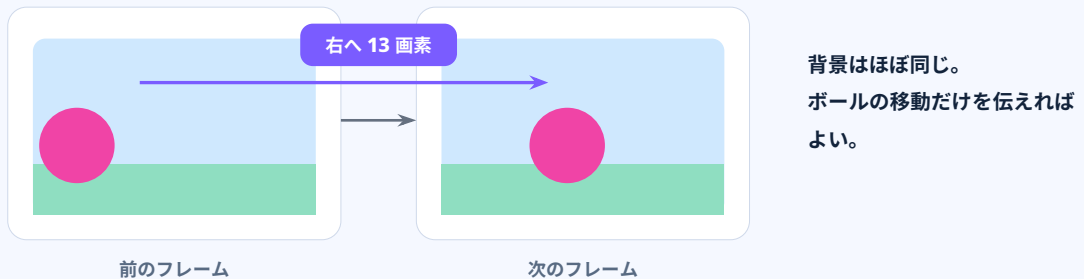
3.1 近くの画素は、たいてい似ている

青空の中の画素は、とんでも青っぽいことが多いでしょう。顔のほっぺの中も、急に真っ黒へ変わることは多くありません。この空間的な似かよいを使うと、「全部の色」を書かずに「となりとの差」だけを書けます。



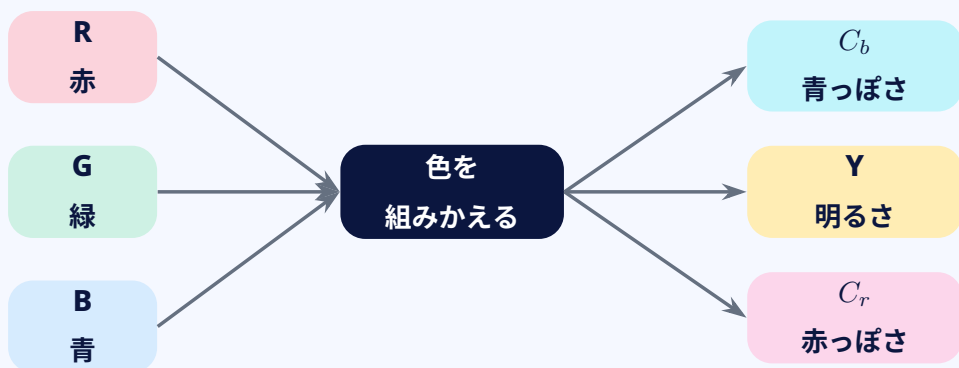
3.2 前のフレームと、たいていよく似ている

動画では、1/30 秒や 1/60 秒で世界が全部入れかわるわけではありません。人やボールが少し移動するだけなら、前のフレームから持ってきて、**どちらへ何画素動いたか**を書けば済みます。これが**時間的な似かよい**です。



3.3 人の目は、明るさにとても敏感

色は赤・緑・青（RGB）でも表せますが、映像圧縮では、明るさ Y と色み C_b, C_r に分けて考えることがよくあります。人の目は細かな明るさの形には敏感ですが、細かな色の違いには少し鈍いからです。



ちょっと数学

明るさを作る考え方の一例は

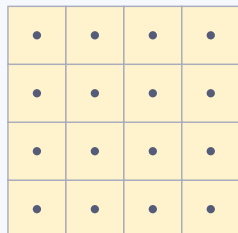
$$Y = 0.2126R + 0.7152G + 0.0722B$$

です。緑 G の重みが大きいのは、人の明るさの感じ方に合わせているためです。実際の映像規格では、色の範囲や丸め方なども細かく決めます。

3.4 4:2:0 は、色のメモを間引く方法

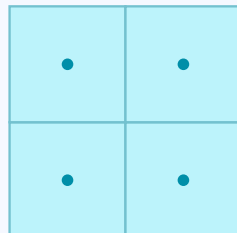
代表的な4:2:0では、明るさ Y は各画素に持たせ、色み C_b, C_r は 2×2 画素で共有します。細かな形は明るさで残し、色のメモを少なくする作戦です。

Y : 明るさ



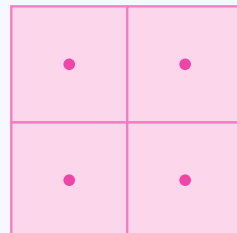
16 個の明るさメモ

C_b : 色み



4 個の色メモ

C_r : 色み



4 個の色メモ

動画圧縮が見つかる 3 種類の「省けるもの」

種類	何が似ている？	主な作戦
空間方向	同じフレームの近くの画素	となりから予測
時間方向	前後フレームの物体や背景	動きを予測
知覚方向	人が気づきにくい細かな色や成分	間引く・丸める

ここだけ覚えよう

圧縮とは、情報をめちゃくちゃに消すことではありません。**予想できる部分を何度も書かず、予想と違うところを中心に書く**ことです。

ミニクイズ

真っ青な空の 100 個の画素を全部別々に書く代わりに、「最初は青、あとはほぼ同じ」と書けそうです。これは 3 つのどの似かよい？

答え：空間的な似かよいです。



3つの「省ける」を組み合わせる

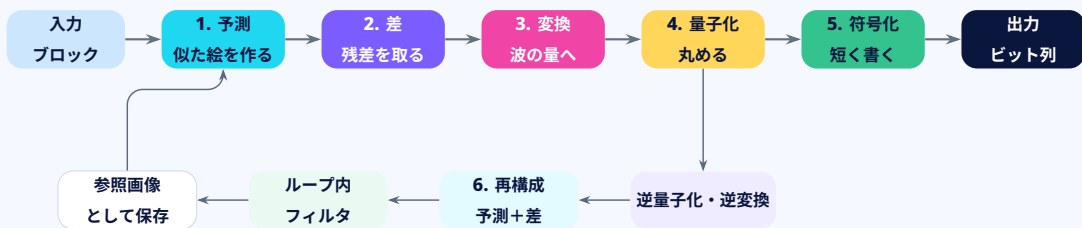
MISSION 4

コーデック工場の 6 つの工程

全体の見取り図

VVC の中には多くの道具がありますが、中心の流れは「予測 → 差 → 変換 → 量子化 → 符号化 → 再構成」です。

4.1 エンコーダの中を、上から見てみよう



ほんとうの話

エンコーダは、デコーダがあとで作る画像と同じ再構成画像を自分の中でも作ります。そうしないと、次のフレームを予測するときに、おたがいが別の絵を見てしまうからです。

4.2 工程 1：予測する

今から圧縮したいブロックに似たものを、**同じフレームの近く**または**前後のフレーム**から探します。ぴったりなら、それだけでかなり説明できます。



4.3 工程 2：予測との差（残差）を取る

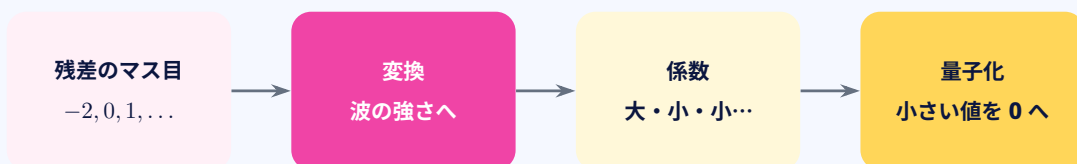
本物を x 、予測を p とすると、残す差 e は

$$e = x - p$$

です。予測が上手なら、 e は 0 や小さな数ばかりになります。小さな数や同じ数が続くデータは、短く書きやすくなります。

4.4 工程 3・4：変換して、丸める

残差をそのまま保存せず、**変換**で「なめらかな変化」「細かな変化」の量へ並べかえます。そして、細かな成分を中心に**量子化**で丸めます。ここが、ファイルをぐっと小さくする大きな場所です。



4.5 工程 5：よく出る記号を短く書く

0 がたくさん出るなら、0 に短い符号を割り当てます。めずらしい値は少し長くても、全体が短くなれば成功です。VVC では **CABAC** という文脈適応型の算術符号化が使われます。

4.6 工程 6：自分でも再生して、次の予測に使う

量子化で丸めたあと、逆向きの計算をして画像を作り直します。境目をなめらかにしたり、誤差のくせを直したりするフィルタを通し、次のフレームの予測に使います。

ここだけ覚えよう

圧縮の合言葉は、「予測して、差だけを、変身させて、丸めて、短く書く」です。

ミニクイズ

エンコーダの中に、なぜデコーダと同じ「再構成」の道がある？

答え：次の予測で、エンコーダとデコーダが同じ参照画像を見るためです。

MISSION 5

引き算から始める、圧縮の数学

数式ミッション

数式は魔法の呪文ではありません。「同じことを短く正確に書く言葉」として、1行ずつ読み解きます。

5.1 まずは1画素：本物－予測＝残差

本物の明るさが152、予測が150なら、送るべき差は2です。

$$\begin{array}{ccccc} \text{152} & - & \text{150} & = & \text{2} \\ \text{本物 } x & & \text{予測 } p & & \text{残差 } e \end{array}$$

ちょっと数学

$$e = x - p$$

左辺の e は「残す差」。右辺は「本物 x から予測 p を引く」と読みます。デコーダは逆に

$$\hat{x} = p + \hat{e}$$

として再構成します。帽子 $\hat{}$ は「圧縮後に作り直した値」という目印です。

5.2 4×4 画素でも、やることは同じ

ブロック全体では、画素を表のように並べた**行列**として引き算します。

本物 X		予測 P		残差 E
$\begin{bmatrix} 52 & 55 & 61 & 66 \\ 51 & 54 & 60 & 65 \\ 50 & 53 & 58 & 63 \\ 49 & 52 & 57 & 62 \end{bmatrix}$	—	$\begin{bmatrix} 52 & 55 & 60 & 64 \\ 51 & 54 & 59 & 63 \\ 50 & 53 & 58 & 62 \\ 49 & 52 & 57 & 61 \end{bmatrix}$	=	$\begin{bmatrix} 0 & 0 & 1 & 2 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

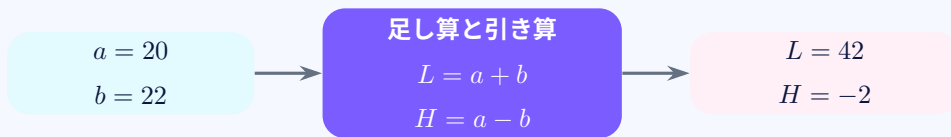
残差には 0・1・2 しかありません。元の 50～66 という数を全部書くより、ずっと規則的になりました。

ここだけ覚えよう

良い予測ほど、残差の多くが 0 に近づきます。**0** が多いほど、あとで短いビット列にしやすいのです。

5.3 変換は「並び方」を変えて、エネルギーを集める

変換は、情報を消す操作ではありません。残差を、別のものさしで測り直して並べかえます。まず 2 個だけで考えましょう。



大きな「平均っぽさ」と小さな「違い」に分かれた

元に戻すときは

$$a = \frac{L + H}{2}, \quad b = \frac{L - H}{2}$$

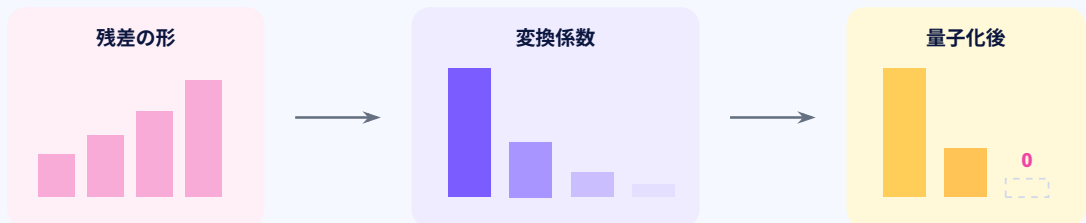
です。 $L = 42, H = -2$ を入れると、ちゃんと 20 と 22 に戻ります。

ちょっと数学

2 次元ブロックの変換は、まとめて

$$C = TET^T$$

と書けます。 E は残差、 T は変換のルール、 T^T は向きを入れかえた行列、 C は変換後の係数です。実際の VVC では、整数で正確に計算できるように設計された変換を使います。

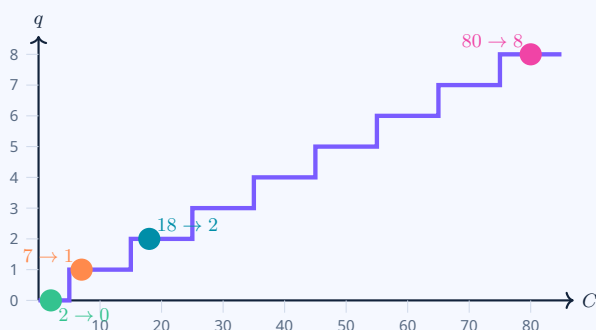


5.4 量子化は「ものさしの目を粗くする」

変換係数 C を、量子化の刻み幅 Δ で割って丸めます。

$$q = \text{round}\left(\frac{C}{\Delta}\right)$$

たとえば $\Delta = 10$ なら、係数 $[80, 18, 7, 2]$ は $[8, 2, 1, 0]$ になります。



小さな係数が 0 になる。
0 が続けば、符号化でとても短く書ける。

まちがえやすい

変換だけなら元へ正確に戻せます。画質が少し変わる主な場所は量子化です。刻み幅 Δ を大きくすると小さくなりやすい一方、誤差も大きくなりやすくなります。

5.5 デコーダは逆向きに進む

量子化された値から係数を作り直す一番単純な考え方は

$$\hat{C} = q\Delta$$

です。先ほどの $[8, 2, 1, 0]$ は $[80, 20, 10, 0]$ になります。もとの $[80, 18, 7, 2]$ とは少し違いますが、逆変換して予測と足せば、見た目の近い画素へ戻せます。

ここだけ覚えよう

予測で差を小さくし、変換で大切な成分を集め、量子化で細かな成分を丸める。これが圧縮数学の中心です。



小さな係数を 0 にして、短く書きやすくする

MISSION 6

VVC のすごさ 1：形に合わせて細かく分ける

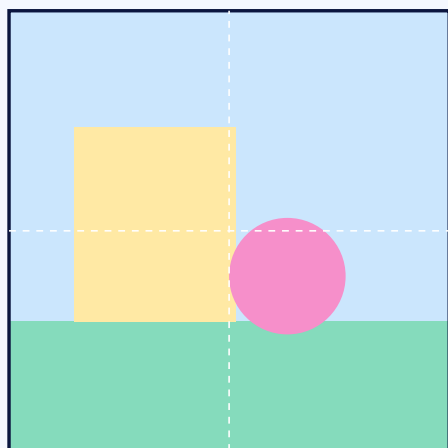
ブロック分割

空・顔・文字・草むらでは、ちょうどよいブロックの形が違います。VVC は四角形を多彩に切り分けます。

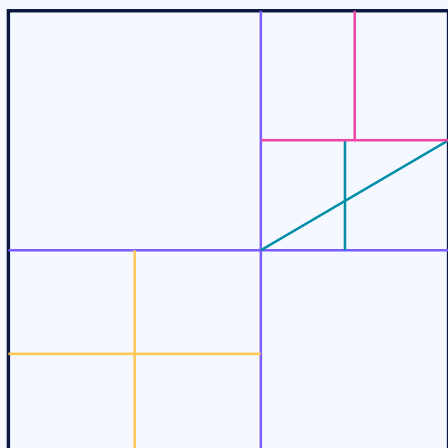
6.1 最初は大きな CTU から

VVC は画像を大きな処理単位に分けます。代表的な最大サイズは**128×128 画素**です。この大きな四角を**CTU (Coding Tree Unit)**と呼び、内容に合わせて小さく分けます。

ひとつの 128×128 CTU (例)



形に合わせた分割

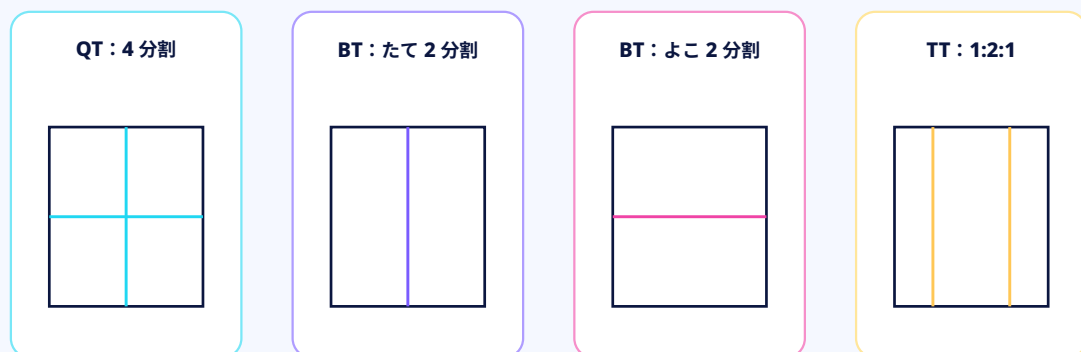


ほんとうの話

大きななめらかな空は大きなブロックのまま、細かな文字や輪郭の近くは小さく分けると効率がよくなります。VVC の CTU は最大 128×128 画素で、HEVC の代表的な最大 64×64 より大きくできます。

6.2 QT・BT・TT という切り方

VVC の分割では、4 分割の QT (Quad Tree) に加え、2 分割の BT (Binary Tree)、1:2:1 の 3 分割をする TT (Ternary Tree) を組み合わせます。この仕組みを MTT (Multi-Type Tree) と呼びます。



もっと知りたい人へ

実際には、すべての場所で何回でも自由に切れるわけではありません。ブロックの大きさ、深さ、予測の種類などに応じて制限があります。それでも、QT だけより多くの形を選べるのが VVC の強みです。

6.3 エンコーダは「候補」をくらべる

小さく分ければ予測がぴったり合いやすくなります。しかし、分け方を伝えるビットが増え、計算も増えます。大きすぎれば説明は少なくとも、予測誤差が増えます。そこでエンコーダは、**画質とビット数の合計点**をくらべます。

ちょっと数学

代表的な評価式は

$$J = D + \lambda R$$

です。

- D ：もとの画像との違い（ひずみ）。小さいほどきれい。
- R ：必要なビット数。小さいほど軽い。
- λ ：画質と軽さのどちらをどれだけ重く見るか。

J が小さい候補を選ぶ考え方を、**レート・ディストーション最適化**と呼びます。

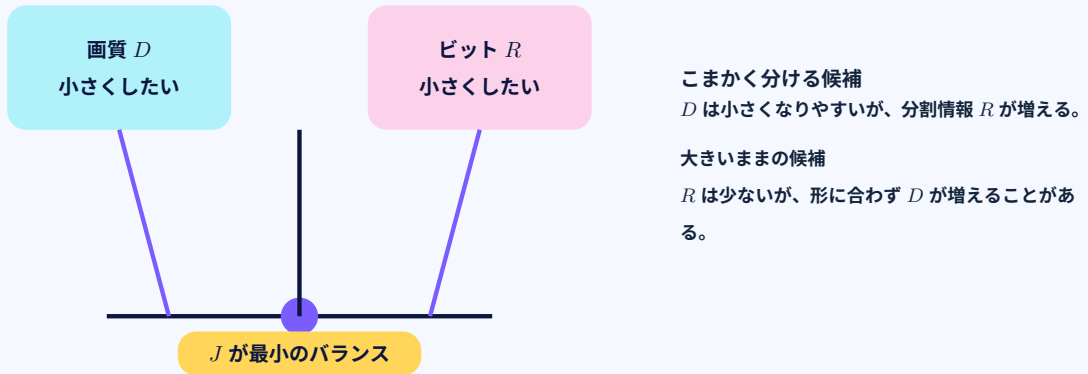


表6.1: 同じ場所を 3 つの分け方で試した例

候補	D	R	$J = D + 2R$
分けない	82	12	106
たて 2 分割	48	20	88
4 分割	34	31	96

この例では、たて 2 分割が勝ちです。4 分割は一番きれいでも、分割を伝えるビットが多すぎました。

ここだけ覚えよう

VVC は「とにかく細かくする」のではなく、**その場所にちょうどよい形と大きさ**を選びます。

MISSION 7

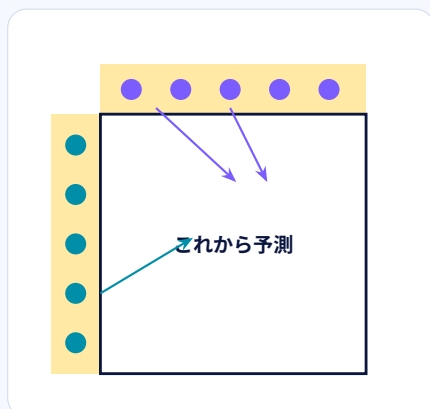
VVC のすごさ 2：予測の引き出しが多い

予測の二刀流

同じフレームから予測する**イントラ予測**と、別フレームから予測する**インター予測**を使い分けます。

7.1 イントラ予測：となりの画素から中を想像する

まだ復号していないブロックの中を、上や左にある復号済み画素から予測します。なめらかな面、たて線、横線、ななめ線など、画像の方向に合う予測を選びます。



上と左は、もう知っている。
そこから線の向きや明るさの変化を伸ばして、中を予測する。

VVC の基本的なイントラ予測モードは、**Planar** (なめらか)、**DC** (平均に近い一定値) と、**65 方向の角度予測**を合わせた 67 種類です。



67 種類

Planar 1

DC 1

角度 65

長方形ブロックでは、方向を読み替える仕組みもある。

7.2 MIP：少ない参照画素から、行列で面を作る

MIP (Matrix-based Intra Prediction) は、上や左の参照画素を少なくまとめ、学習済みではない固定の行列計算でブロックの予測を作る道具です。複雑な面や小さなブロックで役立つ候補になります。

ちょっと数学

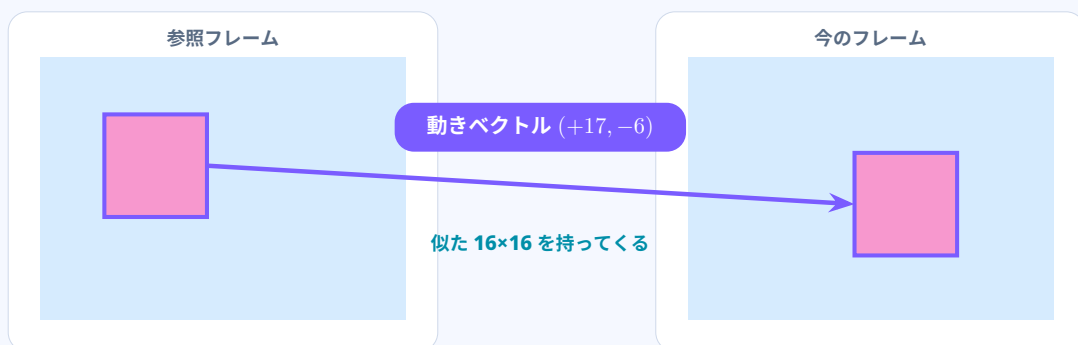
単純化して書くと

$$p = Ar + b$$

です。 r は周りの画素を並べた列、 A は規格で決まった係数の表、 b は調整値、 p が予測ブロックです。ここでの A は映像ごとに AI が学習するものではありません。

7.3 インター予測：前後のフレームから動きを探す

別のフレームに似たブロックがあれば、その場所のずれを**動きベクトル**として送ります。

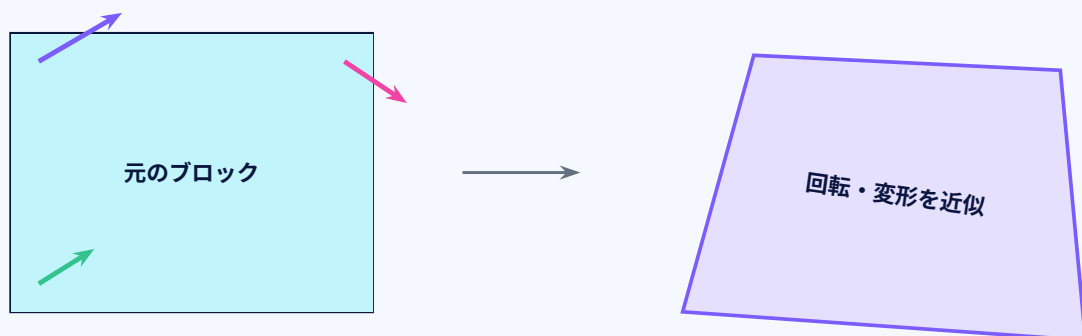


ほんとうの話

動きは必ず整数画素ぶんとは限りません。VVC は画素と画素の間の位置も補間して、より細かな動きを表せます。また、2 枚の参照フレームを混ぜる双方向予測も使えます。

7.4 平行移動だけでなく、回転や拡大も近似する

近づく車や回る看板は、ブロック全体が同じ方向へ動くだけではありません。VVC の **アフィン動き補償** は、複数の制御点の動きから、位置ごとに少し違う動きを作ります。



2次元のアフィン変換を単純化して書くと、座標 (x, y) を

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

へ動かします。回転・拡大縮小・せん断・平行移動を組み合わせて表せます。

7.5 予測候補を、賢く使い回す

近くのブロックの動きベクトルが似ていれば、その候補を使い回せます。VVC には、マージ候補、AMVP、MMVD、対称 MVD、三角形に近い分割を使う予測など、多くの選択肢があります。全部を同時に使うのではなく、エンコーダが必要な候補を試します。

もっと知りたい人へ

予測の候補が多いほど、よく当たる可能性は上がります。しかし、すべてを試すと時間がかかります。実用エンコーダは、見込みの薄い候補を早くやめるなど、探索を省く工夫をします。

ここだけ覚えよう

VVC の予測は、**同じ画面の方向と前後画面の動き**を細かく表せます。予測が当たるほど、残差は小さくなります。

予測の引き出しを選ぶ

イントラ

同じ画面の上・左から

インター

前後画面の動きから

アフィン

回転・拡大も近似

いちばん残差が小さくなる候補を採用

MISSION 8

VVC のすごさ 3：残差に合う変換と、短い符号

最後のひとしぼり

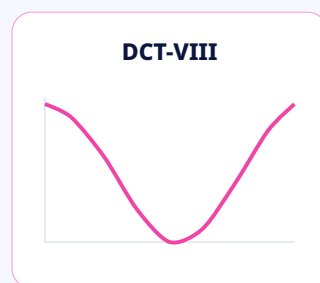
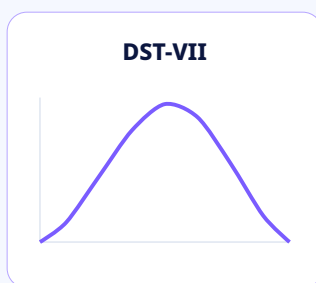
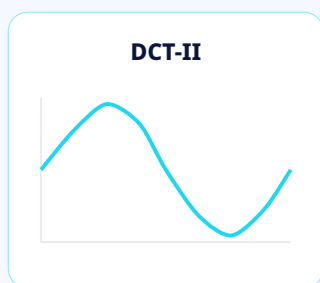
残差の形に合う変換を選び、不要な細部を 0 へ集め、出やすい記号を短く書きます。

8.1 MTS：変換にも選択肢がある

残差がなめらかに広がる場合と、片側の境界から急に変わる場合では、合う変換が違います。VVC の **MTS (Multiple Transform Selection)** では、主に次の変換の組み合わせを候補にできます。

変換

DCT-II	幅広い残差に使う基本選手	なめらかな波
DST-VII	境界側から変化する形に合うことがある	片側が強い波
DCT-VIII	DST-VII と別向きの性質を持つ候補	反対側が強い波



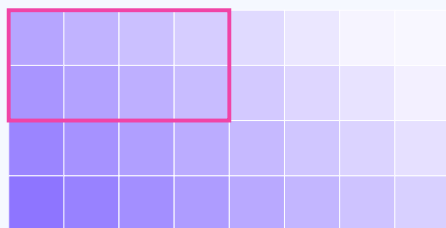
ほんとうの話

「この変換がいつも一番」ではありません。エンコーダは残差の形と必要ビット数を見て選びます。大きなブロックでは計算量を抑えるための制限もあります。

8.2 LFNST：低い周波数どうしを、もう一度まとめる

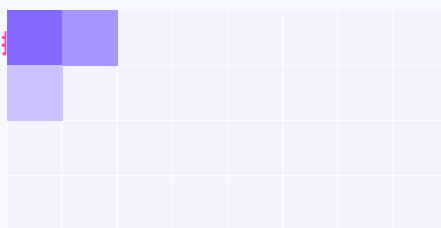
最初の変換後、左上に集まりやすい低周波係数だけを小さな行列でもう一度変換するのが**LFNST (Low-Frequency Non-Separable Transform)**です。イントラ予測残差の方向性を、さらに集められることがあります。

1 回目の係数



LFNST 後

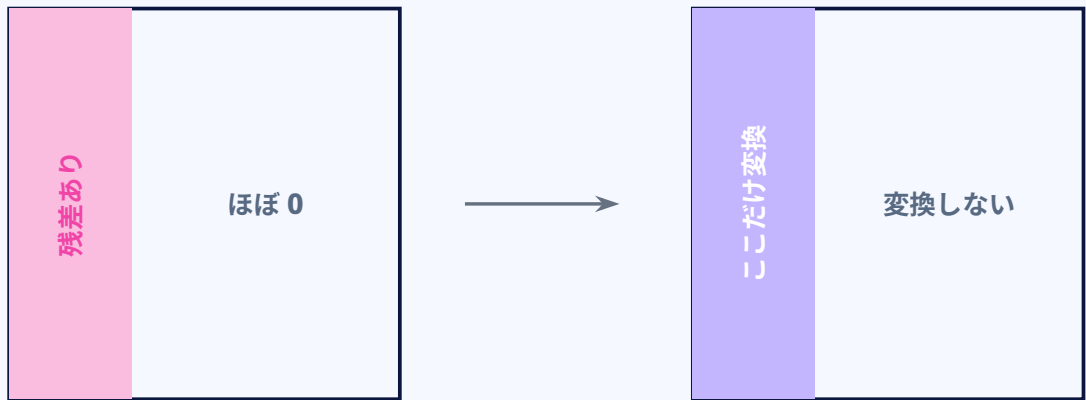
左上だけを再変換



大きな値がさらに少数へ集まる

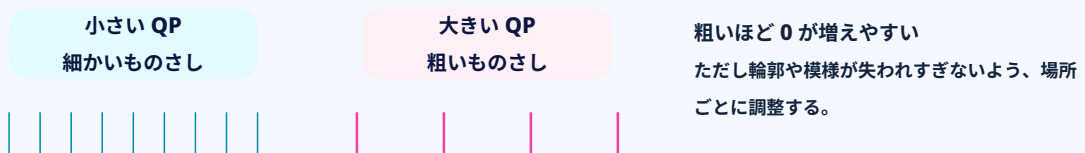
8.3 SBT：残差がある部分だけを変換する

予測がほぼ当たり、ブロックの片側にだけ残差があるときは、全部を変換する必要がないことがあります。**SBT (Subblock Transform)** は、ブロックの一部へ変換を適用する候補です。



8.4 量子化：0を増やすが、誤差も管理する

量子化パラメータ**QP**が大きいほど、おおむね量子化の刻みも大きくなり、ビット数は減りやすく、誤差は増えやすくなります。VVC には、前後の係数の状態も見ながら丸め方を選ぶ**依存量子化**があります。



8.5 CABAC：出やすさを予想して、短く書く

CABACは、周りの状態から「次に 0 が出そう」「このフラグは 1 が少なそう」と確率を予想し、その確率に合う長さでビット列を作ります。

ちょっと数学

ある記号が出る確率を p とすると、その記号が持つ理想的な情報量は

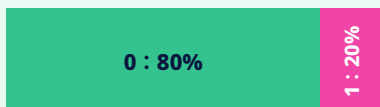
$$I = -\log_2 p$$

ビットです。たとえば

$$p = \frac{1}{2} \Rightarrow I = 1, \quad p = \frac{1}{4} \Rightarrow I = 2, \quad p = \frac{1}{8} \Rightarrow I = 3.$$

めずらしい記号ほど、説明に多くのビットが必要になります。

ある場所での予想



0 を短く、1 を長めに。

別の場所では確率モデルが変わる。これが「文脈適応」の考え方。

まちがえやすい

「0 は必ず 1 ビット、1 は必ず 3 ビット」のように固定するわけではありません。算術符号化は、多くの記号をまとめて、確率に近い平均ビット数へ近づけます。

ここだけ覚えよう

VVC は、**残差に合う変換**で値を集め、**量子化**で 0 を増やし、**CABAC** で出やすさに合う短い表現へ変えます。



値を集める → 0を増やす → 短い記号へ

MISSION 9

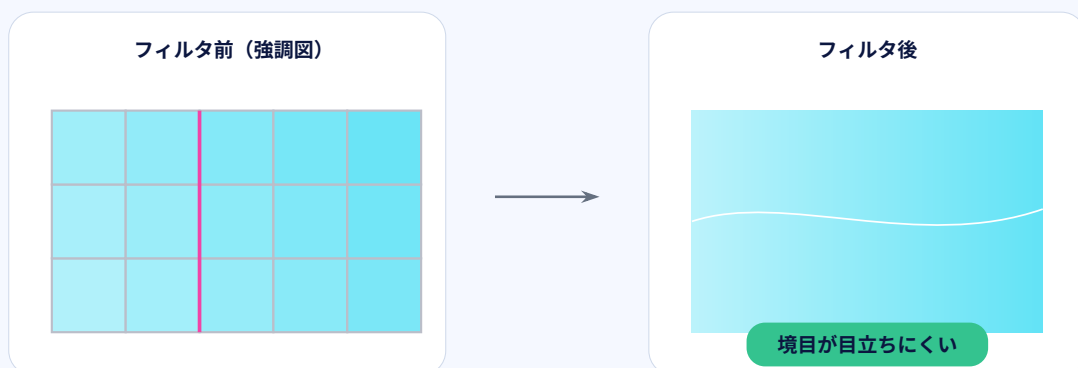
VVC のすごさ 4：再生画像をみがき、いろいろな映像に対応する

仕上げと万能性

圧縮で生まれた誤差の「くせ」を直し、次の予測にも使いやすい画像へ整えます。

● 9.1 なぜフィルタが必要なの？

ブロックごとに予測・変換・量子化すると、境目が四角く見えたり、輪郭の近くに小さな波模様が出たりします。これらを減らす道具を**ループ内フィルタ**と呼びます。「ループ内」なのは、きれいにした画像を次の予測の参照にも使うからです。



9.2 3 段階の代表的な仕上げ



道具	何を覚える？	何をやる？
DBF	ブロック境界の左右・上下	不自然な段差を弱める
SAO	再構成誤差の統計的なくせ	画素をいくつかの種類に分け、オフセットを足す
ALF	局所的な方向や模様	選んだ係数で適応的にフィルタする

ほんとうの話

ALF (Adaptive Loop Filter) は、VVC で強化された重要な道具です。エンコーダはフィルタ係数などを送り、デコーダは同じ処理をします。使わないほうがよい場所では、オフにもできます。

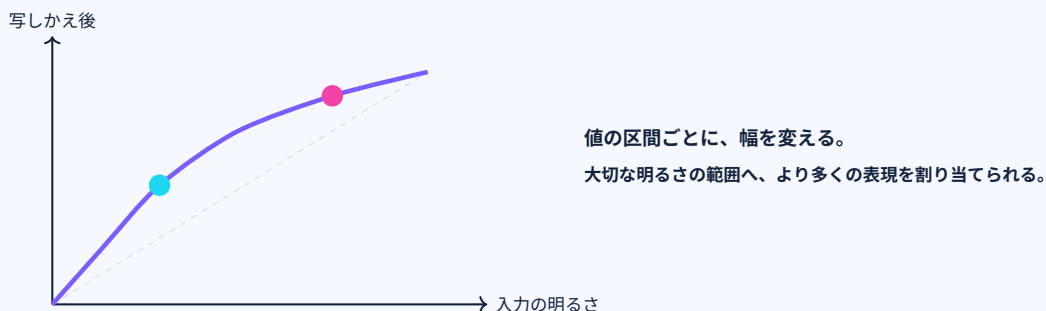
9.3 CC-ALF：明るさを手がかりに、色を直す

色の輪郭は、明るさの輪郭と似ていることがあります。**CC-ALF (Cross-Component ALF)** は、明るさ成分の情報を手がかりにして、色差成分を補正します。



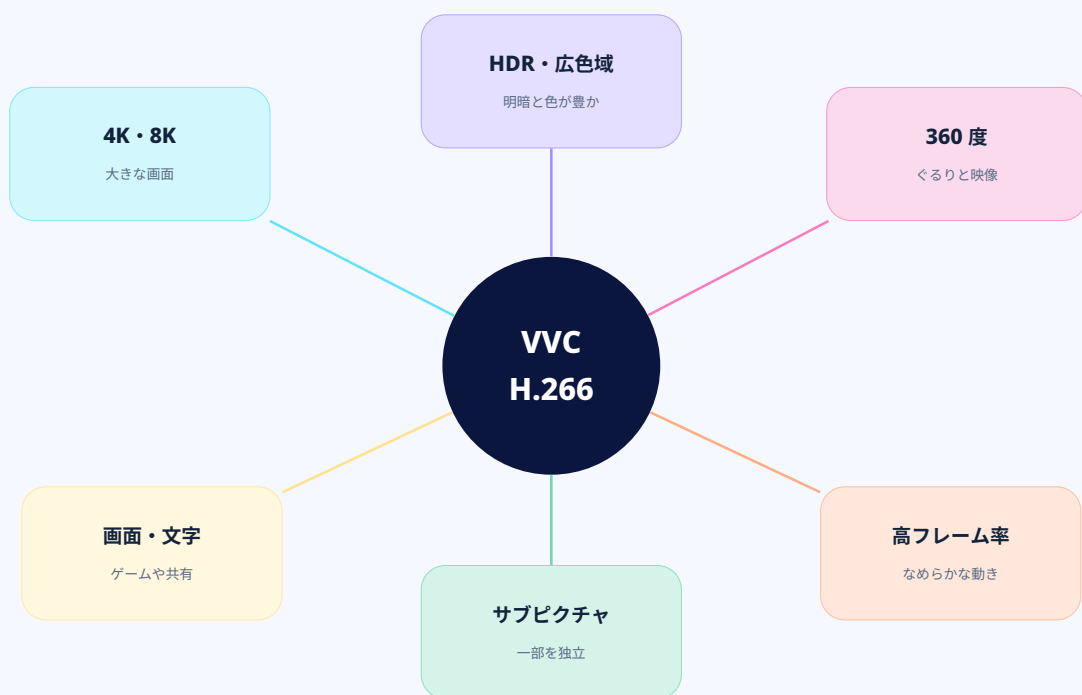
9.4 LMCS：明るさの使い方を映像に合わせる

LMCS (Luma Mapping with Chroma Scaling) は、明るさの値を映像に合う形へ写しかえ、色差の残差も連動して調整する道具です。暗い部分・明るい部分のどこへ細かさを配るかを工夫できます。



9.5 「Versatile」は、いろいろな映像に使えるという意味

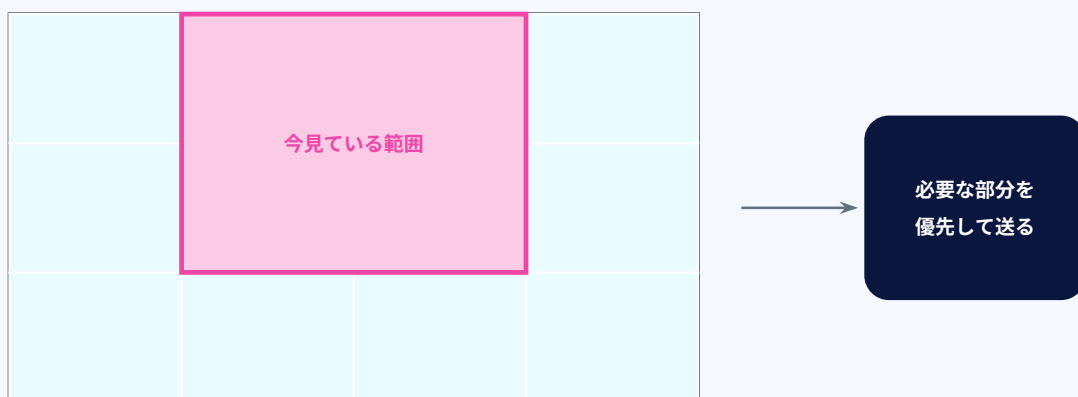
VVC の V は **Versatile**（多用途な、融通のきく）です。高解像度だけでなく、明暗の幅が広い映像、360 度映像、画面共有のような人工的な絵など、幅広い用途を意識して設計されています。



9.6 サブピクチャ：画面の一部をひとまとまりとして扱う

大きな 360 度映像では、視聴者が見ている方向だけを高品質で送る工夫が役立ちます。VVC の **サブピクチャ** は、画面の一部を独立に扱いやすくする仕組みです。

VVC のすごさ 4：再生画像をみがき、いろいろな映像に対応する



まちがえやすい

コーデックだけで配信が完成するわけではありません。ネットワーク、コンテナ、プレーヤー、端末のデコーダなどが、VVC に対応してはじめて再生できます。

ここだけ覚えよう

VVC は、フィルタで参照画像をみがくと同時に、多様な映像の形・明るさ・用途へ対応できる道具を持っています。

再構成画像をみがく流れ



MISSION 10

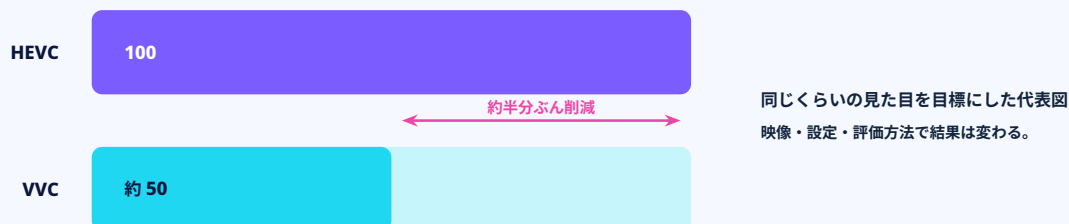
VVC はどれくらい小さくできる？

数字の読み方

「50% 小さい」は魔法の保証ではありません。何と比べ、どの画質で、どんな映像かを確認します。

10.1 代表的な目標：HEVC の約半分のビットレート

VVC は、同じくらいの見た目の品質なら、前世代の **HEVC / H.265** に比べて**ビットレートをおよそ 50% 減らす**ことを大きな目標として開発されました。公表されている比較でも、内容や設定により幅はありますが、約半分という代表値がよく示されます。



●●● 出典メモ：Fraunhofer HHI の VVC 解説、および VVC 技術概要。文献 [4][5][6]。

まちながえやすい

どんな動画でも必ず半分ではありません。アニメ、スポーツ、暗い映像、すでに圧縮された動画などで結果は違います。エンコーダの品質設定と実装も大きく影響します。

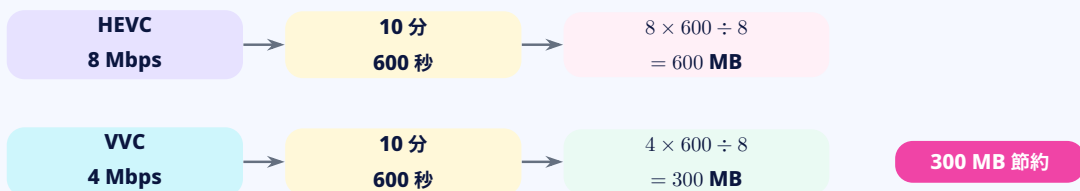
10.2 ビットレートから、だいたいのファイルサイズを計算する

ビットレートは、1 秒間に何ビット使うかです。動画が r Mbps、長さが t 秒なら、音声などを無視した映像サイズの目安は

ちょっと数学

$$\text{サイズ [MB]} \approx \frac{r \times t}{8}$$

です。8 で割るのは、8 ビット = 1 バイトだからです。



10.3 「同じ画質」はどうやって比べる？

一番大切なのは、人が実際に見てどう感じるかです。しかし毎回たくさんの人に評価してもらうのは大変なので、画素の誤差から計算する指標も使います。

ちょっと数学

もとの画素を x_i 、再構成画素を y_i 、画素数を N とすると、平均二乗誤差は

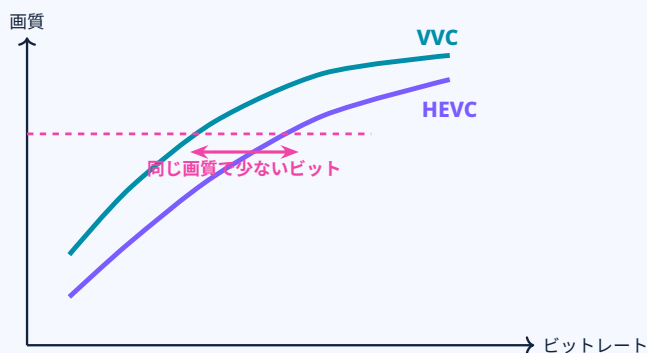
$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2$$

です。差を2乗するので、プラスとマイナスが打ち消しません。MSE が小さいほど、画素値は近いと言えます。

MSE から作る代表的な指標が**PSNR**です。

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right)$$

一般には大きいほど元画像に近いとされますが、人の見た目と完全には一致しません。そのため VMAF などの知覚指標や主観評価も組み合わせます。



レート・ディストーション曲線

左上にあるほど「少ないビットで高画質」。曲線の形は映像ごとに変わる。

10.4 軽さの代わりに、計算は重くなりやすい

VVC は候補が多いので、エンコードは特に重くなりがちです。研究用の参照ソフトウェア比較では、HEVC よりエンコードが約 10 倍、デコードが約 2 倍という例も報告されています。ただし、これは特定の参照実装と設定での値です。製品用ソフトウェアや専用チップでは大きく変わります。

得られるもの

- + 少ない通信量
- + 小さい保存容量
- + 高解像度への余裕

払うもの

- 多い計算
- 対応機器が必要
- 導入・特許の検討

もっと知りたい人へ

参照ソフトウェアは「規格の道具を正しく試す」ためのもので、速度最優先ではありません。実用エンコーダは候補削減、並列処理、SIMD、GPU や専用回路などで高速化します。したがって「VVC は必ず 10 倍遅い」と覚えるのは誤りです。

10.5 3 世代を並べると、進化が見える

表10.1: 代表的な国際映像符号化規格の世代

通称	規格番号	初版の年	さっくりした特徴
AVC	H.264	2003	HD 動画・配信・録画を広く普及させた世代
HEVC	H.265	2013	AVC より高い圧縮効率。4K 時代を支えた世代
VVC	H.266	2020	より柔軟な分割・予測・変換・フィルタ。多様な映像を想定

出典メモ：ITU-T の各勧告と JVET 資料。VVC 初版は 2020 年に完成・承認。文献 [1][3]。

ここだけ覚えよう

VVC の強みは、同じくらいの見た目を、より少ないビットで表せること。ただし、計算量・対応機器・実装条件とのバランスが必要です。

MISSION 11

16 画素で、圧縮を手計算してみよう

実験室

4×4 画素の小さな動画で、動き予測から残差、量子化、再構成までを通して見ます。

● 11.1 前のフレーム：明るい四角が中央にある

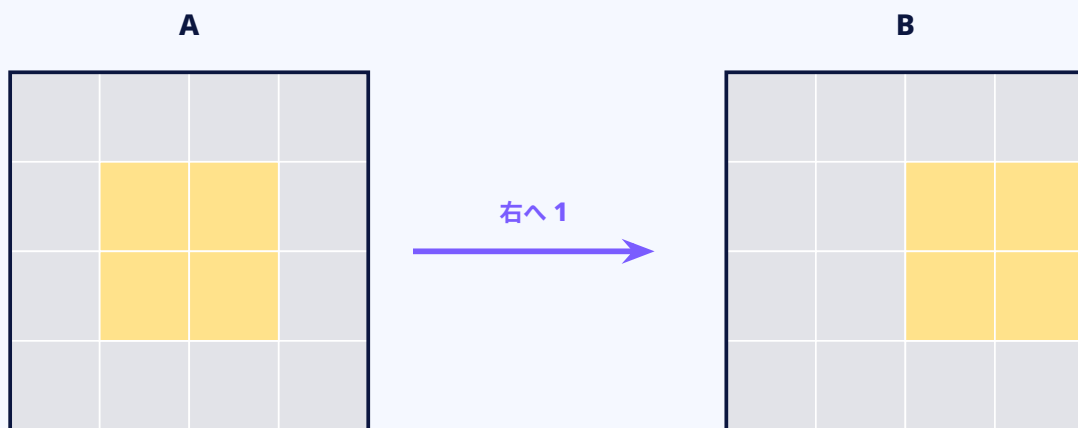
ここでは明るさだけを考え、暗い背景を 20、明るい四角を 90 とします。

フレーム A

20	20	20	20
20	90	90	20
20	90	90	20
20	20	20	20

4×4 = 16 画素各画素は 0~255 の明るさとする

11.2 次のフレーム：四角が右へ 1 画素動いた



フレーム B の 16 個の値をもう一度全部書く代わりに、エンコーダはフレーム A から似た形を探し、**動きベクトル (+1, 0)** を候補にします。

ここだけ覚えよう

(+1, 0) は「横に +1、縦に 0」、つまり右へ 1 画素という意味です。この本では画面座標を単純化しています。

11.3 予測ブロックを作る

A の明るい四角を右へ 1 画素ずらして、B の予測 P を作ります。空いた左端は、ここでは背景 20 で埋めるとします。

予測 P				本物 X			
20	20	20	20	20	20	20	20
20	20	90	90	20	20	90	90
20	20	90	90	20	20	90	90
20	20	20	20	20	20	20	20

=

ぴったり一致！
予測だけで 16 画素の形を説明できた。

11.4 残差を引き算する

$$E = X - P$$

なので、16 画素すべてが 0 です。

残差 E

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

送る中心情報

参照フレーム A

動き (+1,0)

残差は全部 0

実際にはブロック分割や予測モードなどの情報も必要。

ほんとうの話

圧縮前は「20,20,20,...,90」と16個の画素を持ちます。圧縮後の考え方は「前のこの場所を、右へ1動かす。差は0」です。これが時間方向予測の力です。

11.5 完全一致しないとき：1 画素だけ 92 だったら

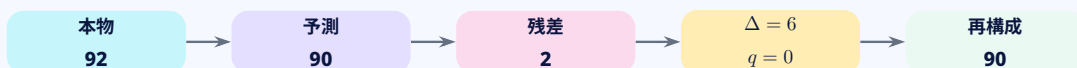
現実の映像では、光やノイズで少し変わります。B の右上側の明るい画素が 90 ではなく 92 だったとします。



11.6 量子化の細かさで、結果が変わる

刻み幅 Δ	量子化 値 (quantized)	量子化 値 (quantized)	量子化 値 (quantized)	再構成 値 (reconstructed)
細かい：2	2	1	2	92 (ぴったり)
粗い：6	2	0	0	90 (2 だけ違う)

粗い量子化では、残差 2 が 0 になり、1 個ぶんの情報を省けます。その代わり再構成値は 92 ではなく 90 です。この小さな違いが目立つかどうかを考えて、エンコーダは設定を選びます。



11.7 この小さな実験に、VVC の全体が入っている

順番	やったこと	VVC の本物では
1	4×4 をひとまとまりにした	CTU から QT/BT/TT で分割候補を選ぶ
2	前フレームから右へ 1 動かした	動き探索、補間、アフィン、双方向なども候補
3	本物ー予測を計算した	$Y \cdot C_b \cdot C_r$ ごとに残差を作る
4	残差を丸めた	変換、MTS、LFNST、量子化を使う
5	予測＋残差で再構成した	DBF、SAO、ALF などを通して参照にする

もっと知りたい人へ

実際の VVC のビットストリームは、動きベクトルをそのまま二進数で書くだけではありません。近くの候補との差、各種フラグ、係数の走査、CABAC の確率モデルなどを組み合わせます。この章は「圧縮の核」に見える大きさへ縮めたモデルです。

ここだけ覚えよう

16 画素のうち 15 個が予測どおりなら、15 個を何度も書く必要はありません。予測と違う 1 個だけを中心に伝えるのが圧縮です。

MISSION 12

まとめ：VVC を 30 秒で説明しよう

最終ミッション

ここまでの知識を一本の物語につなぎ、相手に合わせて短く説明できればクリアです。

12.1 動画がビット列になるまで



ここだけ覚えよう

VVC は、動画を形に合わせて分け、近くや前後から予測し、予測との差を変換・量子化し、出やすさに合わせて短いビット列にするコーデックです。

12.2 VVC の「すごい」を 4 枚のカードで

1

しなやかな分割

QT・BT・TT で形に合わせる

2

多彩な予測

67 イントラモードや高度な動き

3

変換と符号

MTS・LFNST・CABAC

4

仕上げと多用途

ALF・HDR・8K・360 度

● 12.3 3つの長さで説明してみよう

10 秒版

VVC は、動画の似ている部分を予測して、違うところだけを短く保存する新しい映像コーデックです。

30 秒版

動画は大量の画素でできています。VVC は画面を形に合わせて分け、近く画素や前後フレームから予測します。予測との差を変換して丸め、よく出る記号を CABAC で短く書きます。同じくらいの見た目なら、HEVC より少ないビットを目指します。

2 分版の組み立て

動画は大きい → 似ている → 差だけ → 丸める →
短く書く

そのあとに、QT/BT/TT、イントラ/インター予測、MTS、ALF の例を 1 つずつ足します。

説明チャレンジ

1 相手を見る

何を知っている人？

2 長さを決める

10 秒・30 秒・2 分

3 図を 1 つ選ぶ

予測と残差が最強

最後は「予測して、違うところだけを書く」で結べば、話の芯が伝わる。

12.4 よくある質問

Q1. VVC は動画を ZIP にするもの？

いいえ。画素を完全に保つ ZIP とは違い、予測・変換・量子化を使って、見た目を保ちながら細部を丸めるのが中心です。

Q2. デコーダも「一番よい予測」を探す？

基本的に、選んだモードや必要な情報はビットストリームで伝えられます。重い候補探索を主に行うのはエンコーダです。デコーダは決められた手順で再現します。

Q3. 量子化をなくせば最高？

画素は近くなりますが、ビット数が大きくなりやすくなります。用途に合う画質と容量のバランスが必要です。

Q4. VVC なら必ず HEVC の半分？

いいえ。約 50% 削減は代表的な目標・比較結果です。映像、設定、エンコーダ、評価方法によって変わります。

Q5. VVC は AI コーデック？

VVC の標準的な復号手順は、規格で決まった予測・変換・フィルタなどです。エンコーダの探索を AI で助ける実装はありえますが、VVC そのものを「AI だけのコーデック」と呼ぶのは正確ではありません。

12.5 卒業クイズ：全 10 問

問題

1. 動画の 1 枚の写真を何と呼ぶ？
2. コーデックの「CO」と「DEC」は何の略？
3. MP4 は主にコーデック？それともコンテナ？
4. $e = x - p$ の e は何？
5. 変換と量子化のうち、主に情報を失うのはどちら？
6. QT・BT・TT は何を選ぶための道具？
7. 同じフレームの近くから予測する方法は？
8. 前後のフレームから予測するときに使う矢印のような情報は？
9. CABAC は、よく出る記号とめずらしい記号をどう扱う？
10. VVC の正式な規格番号は？

答え

- | | |
|--------------------|--------------------|
| 1. フレーム | 6. ブロックの分け方 |
| 2. Coder と Decoder | 7. イントラ予測 |
| 3. コンテナ | 8. 動きベクトル |
| 4. 残差（予測との差） | 9. 出やすい記号を平均的に短くする |
| 5. 量子化 | 10. H.266 |

MISSION COMPLETE

あなたは **VVC** の圧縮ストーリーを説明できる！

次に動画を見たとき、その裏で「予測して、差だけ」が働いていることを思い出そう

見える

動画・フレーム・画素

✓ クリア

説明できる

予測・残差・量子化

✓ クリア

比べられる

HEVC との効率と計算量

✓ クリア

つながられる

分割からフィルタまで

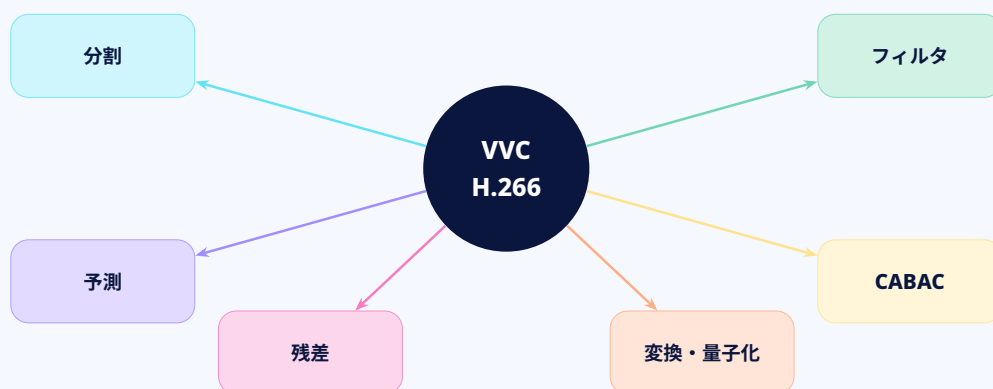
✓ クリア

VVC EXPLAINER LEVEL 1

ことば図鑑

用語	この本での意味
ALF	Adaptive Loop Filter。画像の局所的な形に合わせて再構成画像を整えるループ内フィルタ。
AVC / H.264	VVC より 2 世代前の代表的な映像コーデック。
ビット	0 か 1 の、デジタルデータの最小単位。
ビットストリーム	コーデックの規則で並べられた 0 と 1 の列。
ビットレート	1 秒あたりに使うビット数。Mbps などで表す。
BT	Binary Tree。ブロックをたて、またはよこに 2 分割する方法。
CABAC	文脈から確率を予想し、算術符号化で記号を短く表す方法。
CC-ALF	明るさ成分を手がかりに色差成分を補正するフィルタ。
コーデック	エンコードとデコードの規則・仕組みの組み合わせ。
コンテナ	映像・音声・字幕などを入れる箱。MP4 や MKV など。
CTU	Coding Tree Unit。ブロック分割を始める大きな処理単位。VVC では最大 128×128 画素が代表。
DBF	Deblocking Filter。ブロック境界の不自然な段差を弱める。
デコード	圧縮されたビット列から再生用の映像を組み立てること。
DCT / DST	残差を周波数のような成分へ並べかえる変換の仲間。
エンコード	もとの動画から圧縮されたビット列を作ること。
フレーム	動画を作る 1 枚 1 枚の静止画。
fps	frames per second。1 秒間のフレーム数。
HEVC / H.265	VVC の 1 世代前の代表的な映像コーデック。
イントラ予測	同じフレーム内の復号済み画素から予測する方法。
インター予測	別のフレームから予測する方法。動き補償を使う。
LFNST	低周波係数を小さな行列でもう一度変換する道具。
LMCS	明るさの値を写しかえ、色差残差も連動して調整する道具。
MIP	Matrix-based Intra Prediction。固定行列を使うイントラ予測候補。
MSE	もとの画素と再構成画素の差を 2 乗し、平均した値。
MTS	Multiple Transform Selection。残差に合わせて複数の変換候補から選ぶ仕組み。

用語	この本での意味
MTT	Multi-Type Tree。QT・BT・TTを組み合わせたブロック分割。
動きベクトル	参照フレームのどこから、どちらへどれだけ動いたかを表す情報。
画素／ピクセル	画像を作る小さな色・明るさの点。
PSNR	MSE から計算する画質指標の一つ。一般には高いほど元画像に近い。
予測	周りや前後の映像から、今のブロックに似た画像を作ること。
QP	量子化の細かさに関するパラメータ。大きいほど粗くなりやすい。
QT	Quad Tree。ブロックを 4 分割する方法。
量子化	係数を刻み幅で丸め、小さな値を 0 にしやすくする処理。
残差	本物の画素から予測画素を引いた差。
SAO	Sample Adaptive Offset。誤差のくせに合わせて画素へ補正値を足す。
SBT	Subblock Transform。残差がある一部分を中心に変換する候補。
TT	Ternary Tree。ブロックを 1:2:1 の比で 3 分割する方法。
変換	残差を別の成分へ並べかえ、エネルギーを少数の係数へ集める処理。
VVC / H.266	Versatile Video Coding。2020 年に初版が完成した国際映像符号化規格。
YCbCr	明るさ Y と、青・赤方向の色差 C_b , C_r で色を表す方法。
4:2:0	明るさより色差のサンプル数を少なくする代表的な色差サブサンプリング。



ことばは別々でも、全部が「予測して差だけ」の物語につながる

参考資料と、正確さについて

この本の図について

図・表・数値例はすべて本書のために作成した学習用のものです。VVC の規格図をそのまま転載していません。実際のビットストリーム構文や処理条件は、公式規格・技術論文のほうが優先されます。

- [1] ITU-T, *Recommendation H.266: Versatile video coding*. 国際電気通信連合の公式勧告ページ。

[ITU-T H.266 公式勧告ページ](#)

- [2] ISO/IEC 23090-3, *Information technology - Coded representation of immersive media - Part 3: Versatile video coding*. ISO/IEC の対応国際規格。

- [3] ITU-T / ISO/IEC, *Joint Video Experts Team (JVET)*. VVC の共同標準化チーム。初版は 2020 年 7 月に完成し、同年に承認・公開。

[ITU JVET 公式ページ](#)

- [4] Fraunhofer HHI, *An Introduction to Video Compression*. 画素、フレーム、予測、色差サブサンプリングなどの入門解説。

[Fraunhofer HHI 入門ページ](#)

- [5] Fraunhofer HHI, *H.266 / Versatile Video Coding (VVC) および VVC 関連解説*. 代表的な約 50% ビットレート削減目標、8K・HDR・360 度などの用途説明。

[Fraunhofer HHI VVC 公式解説](#)

- [6] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, J.-R. Ohm, "Overview of the Versatile Video Coding (VVC) Standard and its Applications," *IEEE Transactions on Circuits and Systems for Video Technology*, 2021.

DOI: [10.1109/TCSVT.2021.3101953](https://doi.org/10.1109/TCSVT.2021.3101953)

- [7] W. Hamidouche et al., "The Versatile Video Coding Standard: A Review from Coding Tools to Consumers Deployment," arXiv:2106.14245, 2021. CTU、MTT、イントラ予測、MTS、LFNST、フィルタ、参照ソフトウェアの計算量などの確認に使用。

[arXiv:2106.14245](https://arxiv.org/abs/2106.14245)

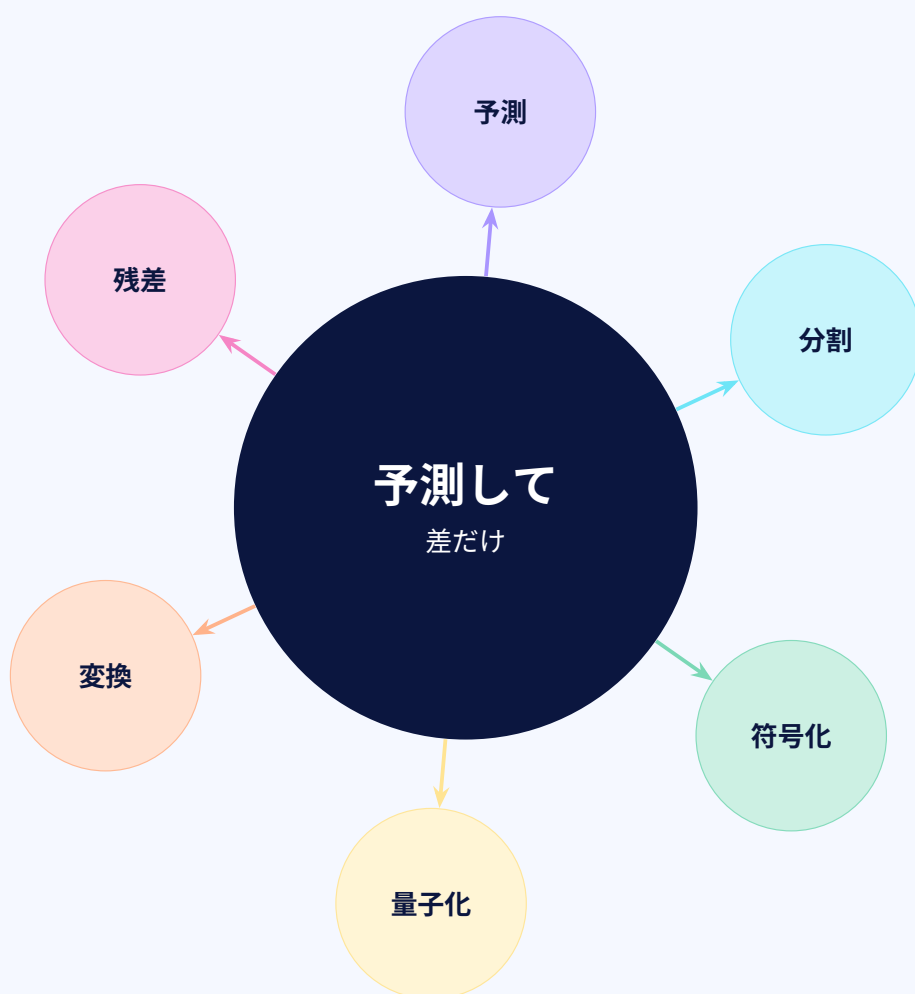
約 50% という数字の読み方

本書の「HEVC より約 50% 少ないビットレート」は、同等の知覚品質を目標にした代表的な説明です。映像内容、解像度、遅延条件、エンコーダ、品質指標、主観評価によって結果は変わります。

数式の単純化

本書の $e = x - p$ 、 $C = TET^T$ 、 $q = \text{round}(C/\Delta)$ などは、中心原理を見せるための一般化・単純化です。実際の VVC では整数変換、スケーリング、クリッピング、予測モード依存の条件、ブロックサイズ制限、色成分ごとの処理、CABAC 構文などが規格で細かく定義されています。

おわりに



動画コーデックは、ただデータを押しつぶす機械ではありません。画像の形を見て、動きを見つけ、人の見え方を考え、数学で「同じものを何度も書かない」工夫を積み重ねた仕組みです。

VVC は、その工夫を多くの選択肢へ広げた規格です。選択肢が増えたぶん、エンコーダはむずかしくなります。しかし、世界中で同じルールを使うからこそ、作ったビット列を別の機械でも再生できます。

ここだけ覚えよう

次に動画を再生したとき、画面の裏では、何百万もの画素が「予測」「差」「変換」「量子化」「符号化」を通して届いていることを思い出してください。

SEE THE PATTERN

PREDICT

SEND THE DIFFERENCE

おしまい。ここからは、あなたが説明する番です。

「動画を小さくする」って、 どういうこと？

答えは、予測して、違うところだけを伝えること。

この本は、VVC / H.266 の分割・予測・変換・量子化・CABAC・フィルタを、オリジナル図解と小さな数式でたどります。

小学生から読める、でも中身は本格派

この本でわかること

- ✓ コーデックとコンテナの違い
- ✓ VVC が HEVC より効率を高める仕組み
- ✓ 残差・変換・量子化の数学
- ✓ QT / BT / TT、67 イントラモード、ALF
- ✓ 4×4 画素で行う手計算実験

表紙名は VCC。正式な技術名は VVC (Versatile Video Coding) / H.266 です。**01011001**